

Sistem Operasi (INF-201)

Pertemuan 11: Implementasi Sistem Berkas (ext4, ZFS & Journaling)

Tim Dosen Sistem Operasi

Program Studi Informatika – Universitas Bengkulu

Semester Ganjil 2026/2027

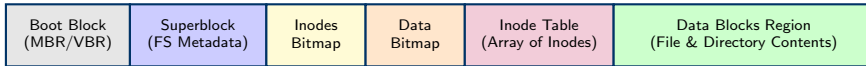
Tanggal Kompilasi: 8 September 2026



Capaian Pembelajaran (Sub-CPMK 11)

- Mahasiswa mampu menganalisis layout internal penyimpanan blok sistem berkas (Superblock, Bitmaps, Inode Table, Data Blocks).
- Mahasiswa memahami akar permasalahan **Crash Consistency** saat terjadi kegagalan daya mendadak.
- Mahasiswa mampu membandingkan mekanisme **Journaling (Write-Ahead Logging)** pada ext4/NTFS dengan **Copy-on-Write (CoW)** pada ZFS/Btrfs/APFS.
- Mahasiswa mampu mengevaluasi fitur keandalan mutakhir: *Instant Snapshots*, *Merkle Tree Checksums*, dan integritas data terhadap *Bit-rot*.

Layout Fisik Partisi Sistem Berkas pada Disk



- **Superblock:** Menyimpan jumlah total inode, jumlah blok, ukuran blok, mount time, dan status kebersihan FS (*clean/dirty flag*).
- **Bitmaps:** Peta bit efisien (1 bit per blok/inode) untuk melacak alokasi bebas vs terpakai.
- **Inode Table:** Menyimpan metadata berkas (permissions, owner, size, timestamps, block pointers).

Masalah Crash Consistency: Dilema 3 Penulisan

Skenario: Menambahkan Blok Baru ke Berkas yang Sudah Ada

OS harus melakukan 3 operasi penulisan terpisah ke disk:

- 1 **Menulis Blok Data Aktual** (D_b) di Data Blocks Region.
- 2 **Memperbarui Inode** (I_b) untuk menambahkan pointer ke blok baru dan memperbesar ukuran berkas.
- 3 **Memperbarui Data Bitmap** (B_b) untuk menandai blok tersebut sudah terpakai.

Apa yang Terjadi Jika Listrik Mati di Tengah-Tengah?

- Hanya D_b tertulis: Data hilang, tidak ada referensi di inode.
- Hanya I_b tertulis: Inode menunjuk ke data sampah (*Garbage Data / Security Leak*).
- Hanya B_b tertulis: Terjadi kebocoran ruang disk (*Space Leak*).

Pendekatan Klasik: `fsck` dan Masalah Skalabilitas

File System Checker (`fsck`)

Program utilitas yang berjalan saat booting untuk memindai seluruh partisi disk dan memperbaiki inkonsistensi struktur metadata.

Tahapan Pemeriksaan `fsck`:

- Cek konsistensi blok dan ukuran inode.
- Cek pathname dan struktur direktori.
- Cek referensi konektivitas hard link.
- Cek kesesuaian alokasi bitmap.

Kelemahan Fatal `fsck`:

Skalabilitas Buruk! Memindai disk 10 TB dapat memakan waktu berjam-jam bahkan berhari-hari saat server me-reboot, mengakibatkan *downtime* yang tidak dapat ditoleransi oleh industri modern.

Solusi Modern 1: Journaling (Write-Ahead Logging)

Prinsip Dasar Write-Ahead Logging (WAL)

Sebelum memperbarui blok data/metadata asli di disk (*In-Place*), OS menulis terlebih dahulu ringkasan niat modifikasi (*intent*) ke dalam area khusus yang disebut **Journal (Log)**.

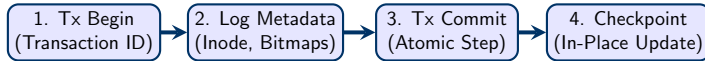
Alur Operasi Normal:

- 1 Catat transaksi ke Journal.
- 2 Tulis Commit Record ke Journal.
- 3 Tulis data asli ke lokasi disk (*Checkpointing*).
- 4 Bebaskan entri Journal (*Free Transaction*).

Alur Recovery Setelah Crash:

- OS hanya membaca area Journal (beberapa MB saja).
- Transaksi yang berstatus **Committed** langsung di-*replay* ke disk asli dalam hitungan detik!
- Transaksi yang belum committed langsung dibuang.

Siklus Transaksi Journaling ext4



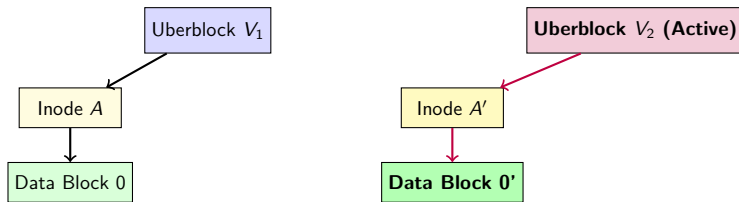
3 Mode Journaling pada Linux ext4:

- **Data Journaling:** Menulis seluruh metadata AND data ke jurnal (Paling aman, paling lambat / 2x write).
- **Ordered Mode (Default):** Data ditulis ke disk terlebih dahulu, baru kemudian metadata dicatat ke jurnal. Menjamin tidak ada pointer inode ke data sampah.
- **Writeback Mode:** Metadata dicatat ke jurnal tanpa menunggu data selesai ditulis ke disk (Paling cepat, risiko data korup).

Solusi Modern 2: Copy-on-Write (ZFS, Btrfs, APFS)

Filosofi Copy-on-Write (Never Overwrite In-Place)

Alih-alih menimpa blok lama yang sedang aktif, sistem berkas CoW selalu menulis data baru ke blok kosong baru. Setelah penulisan sukses, pointer pohon hierarki diperbarui secara atomik menuju blok baru.



Integritas Atomik: Jika terjadi mati listrik saat menulis Data 0', Uberblock V_1 tetap utuh. Crash consistency terjamin 100% tanpa perlu journaling!

Keunggulan CoW: Instant Snapshot & Merkle Tree Checksum

Instant Snapshot (Zero Cost)

- Karena data lama tidak ditimpa, membuat snapshot sistem berkas hanya membutuhkan penyimpanan pointer Uberblock lama (V_1).
- Waktu pembuatan snapshot: **0 milidetik!**
- Hanya memakan ruang disk tambahan saat ada data baru yang dimodifikasi.

Perlindungan Silent Data Corruption

- Fenomena *Bit-rot* (kerusakan magnetik/elektronik di disk).
- ZFS menyematkan cryptographic checksum (SHA-256) pada setiap pointer di pohon Merkle.
- Saat pembacaan data, checksum diverifikasi otomatis → jika korup, di-**self-heal** via mirror paritas!

Tabel Perbandingan Sistem Berkas Modern

File System	Mekanisme Integritas	Keunggulan Utama	Kelemahan Utama
ext4 (Linux)	Journaling (WAL)	Cepat, stabil, kompatibilitas universal	Tidak ada checksum data murni, rentan bit-rot
NTFS (Windows)	Journaling + USN	ACL komprehensif, kompresi native	Overhead metadata tinggi
ZFS (OpenZFS)	Copy-on-Write (Merkle)	Self-healing, pooling terintegrasi, snapshot	Kebutuhan RAM besar (ARC cache)
APFS (Apple)	Copy-on-Write	Sangat optimal untuk NVMe SSD	Khusus ekosistem Apple

Tugas Membaca Mingguan (Reading Assignment)

Persiapan Pertemuan 12: Secondary Storage & RAID

Sebelum mengikuti kuliah minggu depan, mahasiswa wajib membaca:

- **[OSTEP]** Chapter 36: *I/O Devices* (hal. 443–454)
- **[OSTEP]** Chapter 37: *Hard Disk Drives* (hal. 455–470)
- **[OSTEP]** Chapter 38: *Redundant Arrays of Inexpensive Disks (RAID)* (hal. 471–480)
- **[OSTEP]** Chapter 44: *Flash-based SSDs* (hal. 561–574)

Pertanyaan Pemicu Diskusi Mandiri

Mengapa algoritma penjadwalan disk SSTF (Shortest Seek Time First) dapat menyebabkan masalah *Starvation*, dan bagaimana SCAN / C-SCAN mengatasinya?